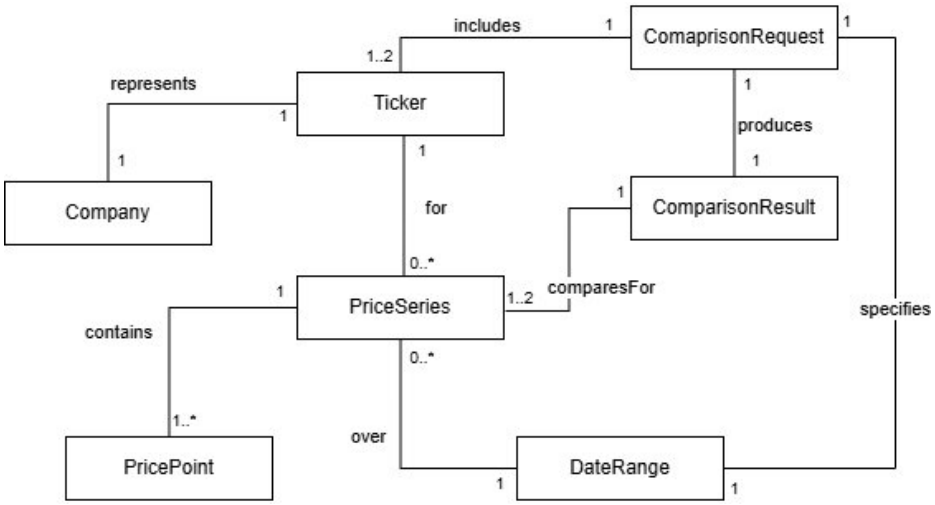
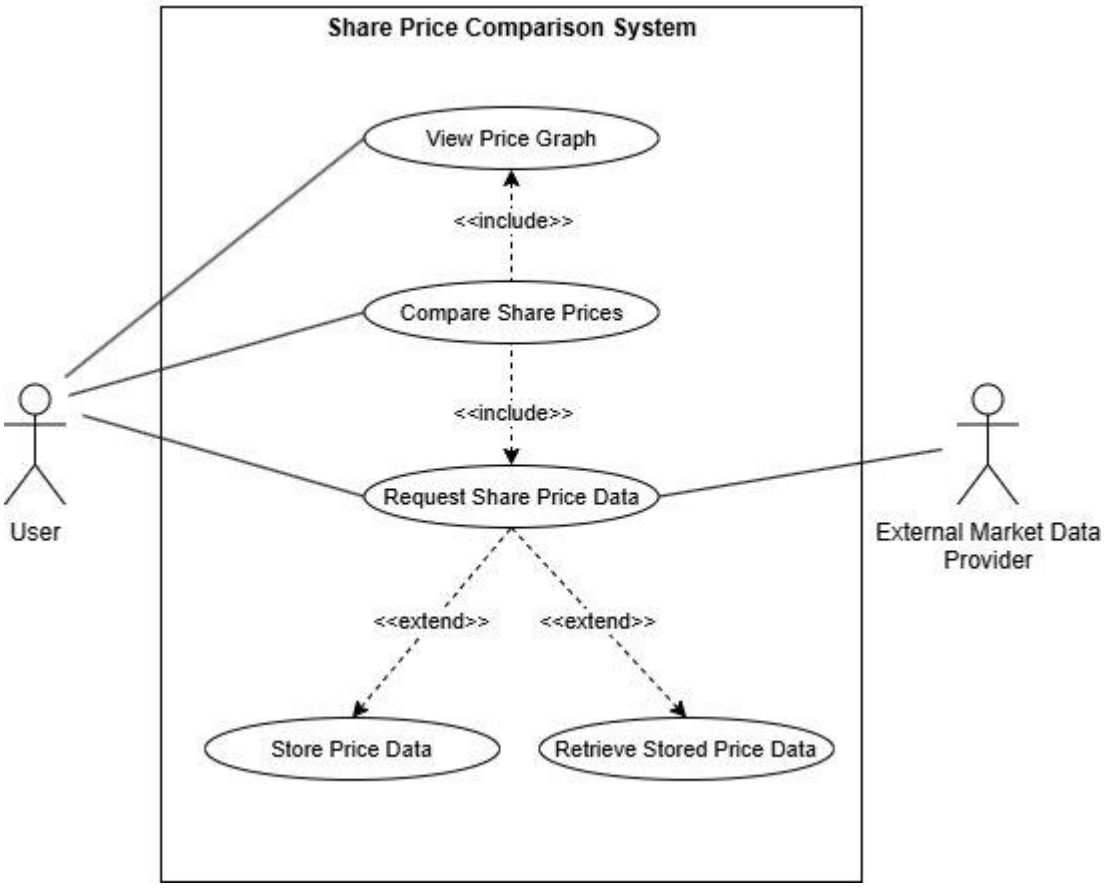
Software Architecture and Design: Sprint 2

1. Business Concept Model



2. Use Case



Software Architecture and Design: Sprint 2

Use Case 1: Request Share Price Data

Actor: User

Goal: Obtain daily share price information for a selected ticker over a specified date range.

Actor Action	System Response
User opens request function	System displays input fields
User enters ticker symbol	System records ticker
User enters date range	System records dates
User submits request	System validates inputs
	System checks date range limit
	System requests data from external provider
	System converts data into PriceSeries and PricePoints
	System stores data locally
	If external fails, system retrieves stored data
	System returns price series
User views result	System displays data

Alternative Scenarios

Invalid Input	• **System rejects request due to invalid ticker or dates** • **User must re-enter valid input**
Date Range Too Large	• System detects range exceeds 2 years • System prompts user to adjust range
External Source Unavailable	• System fails to retrieve data from API • Attempts local retrieval
Local Data Available	• Stored data exists • System returns stored PriceSeries
No Data Available	• External & local retrieval both fail • System informs user

Software Architecture and Design: Sprint 2

Use Case 2: Compare Share Prices

Actor: User

Goal: Compare one or two companies' share prices over a selected date range.

Actor Action	System Response
User opens comparison function	System displays input fields
User enters ticker	System records ticker
User enters date range	System records dates
User submits request	System validates input
	System retrieves data for first ticker
	System retrieves data for second ticker (if present)
	System builds PriceSeries objects
	System creates ComparisonResult
	System sends result to display
User views result	System displays graph

Alternative Scenarios

One Ticker Only	• **System creates a single-series comparison** • **Displays single-company graph**
One Dataset Missing	• One ticker fails retrieval • System shows partial comparison or error
Both Datasets Missing	• No data retrieved • System informs user
Invalid Input	• System rejects invalid ticker or date range

Software Architecture and Design: Sprint 2

Use Case 3: View Price Graph

Actor: User

Goal: View a graphical representation of one or two price series.

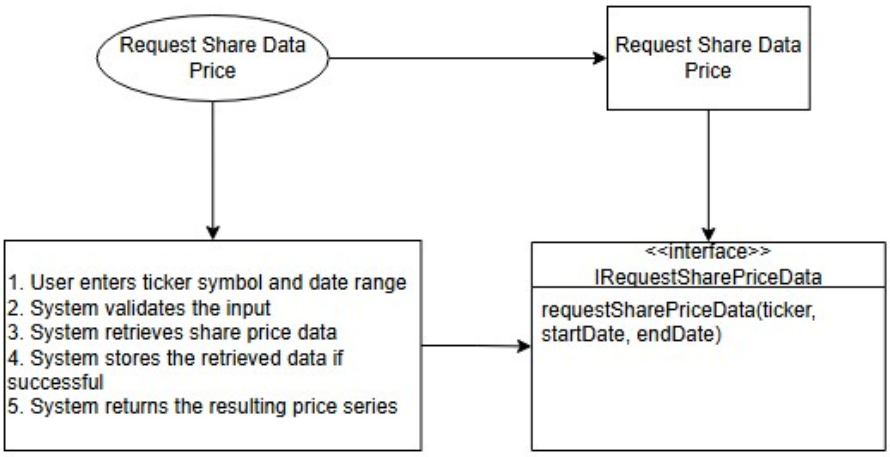
Actor Action	System Response
User requests graph	System receives data to display
	System prepares chart data
	System generates graph
	System labels graph
User views graph	System displays graph

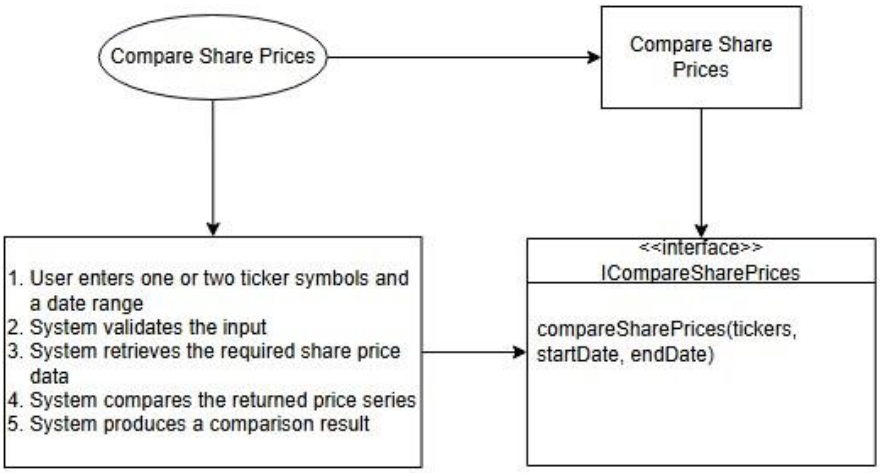
Alternative Scenarios

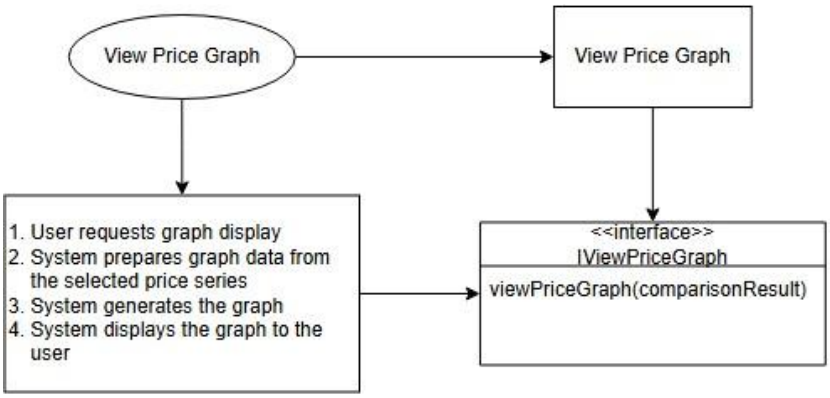
Single Series	• **System displays one line graph**
Incomplete Data	• Missing points result in partial graph or error
Rendering Failure	• Graph cannot be generated • System displays error

Software Architecture and Design: Sprint 2

System Interfaces

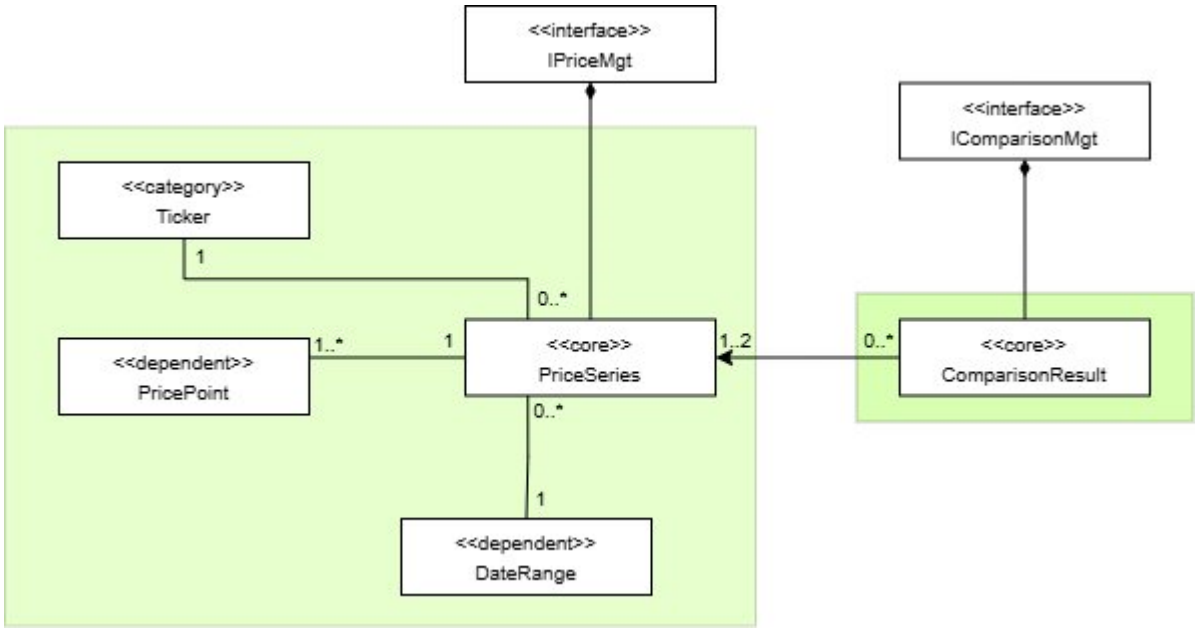




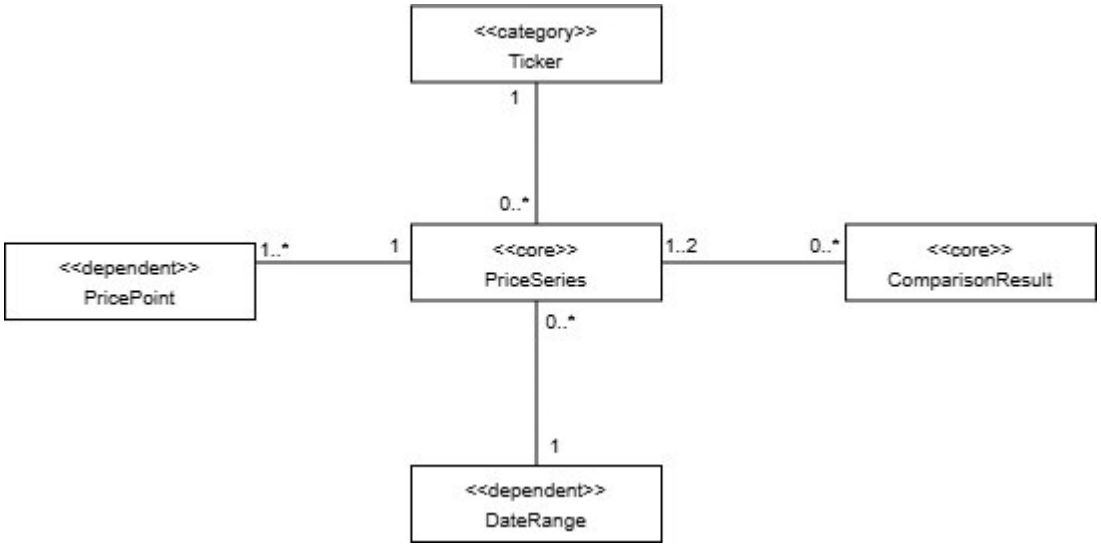


Software Architecture and Design: Sprint 2

Business Type Model

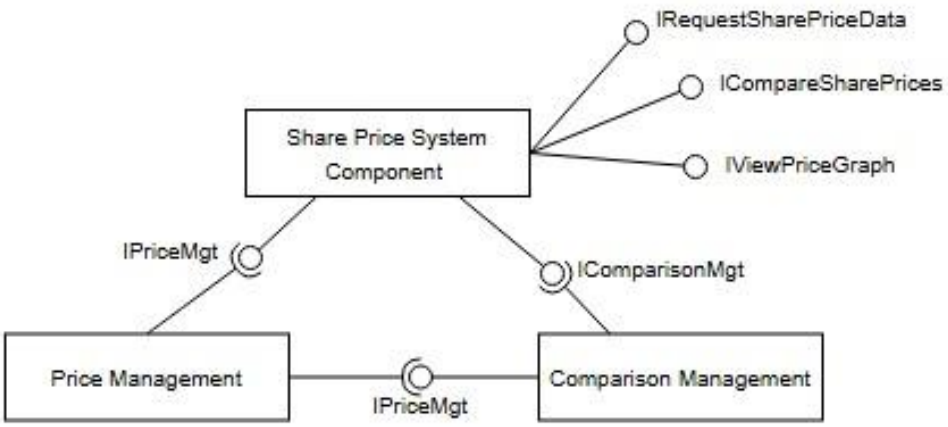


Core/Dependent/Categories



Software Architecture and Design: Sprint 2

Initial System Architecture



The initial system architecture was derived from the previously identified system interfaces and business interfaces. Following the starting assumption from the design process, one business component was allocated per business interface, and one system component was used to support all identified use cases. Therefore there are three main components: System Component, Price Management Component and Comparison Management Component.

The System Component implements the three system interfaces derived from the use cases: IRequestSharePriceData, ICompareSharePrices, and IViewPriceGraph. These interfaces were grouped into a single component because they are all user-facing, belong to the same application-level behaviour, and are likely to evolve together.

The Price Management Component implements the business interface IPriceMgt. This component is responsible for the price-series responsibility region, including the associated Ticker, DateRange, and PricePoint business types. These functions were grouped together because they all relate directly to obtaining, structuring, and managing share price series data.

The Comparison Management Component implements the business interface IComparisonMgt. This component is responsible for creating and managing ComparisonResult objects. It was kept separate from the price management responsibilities because comparison logic forms a distinct business concern, even though it depends on price-series data.

In this architecture, the System Component is the only component that implements more than one interface. Both business components implement only one interface each. A dependency exists between the Comparison Management Component and the Price Management Component though, as a comparison result is constructed from one or two price series.

The initial system architecture is derived at a coarse-grained level from the identified system interfaces and business interfaces. It translates well to the higher-level architecture of the component diagram from Sprint 1 that is refined in the implementation into more concrete components, including the User Interface, Comparison Service, Price Repository, Local Price Store, and Market Data Provider.

Software Architecture and Design: Sprint 2

Business Interfaces

IPriceMgt

This interface is responsible for the price management region centred on PriceSeries, including Ticker, DateRange, and PricePoint.

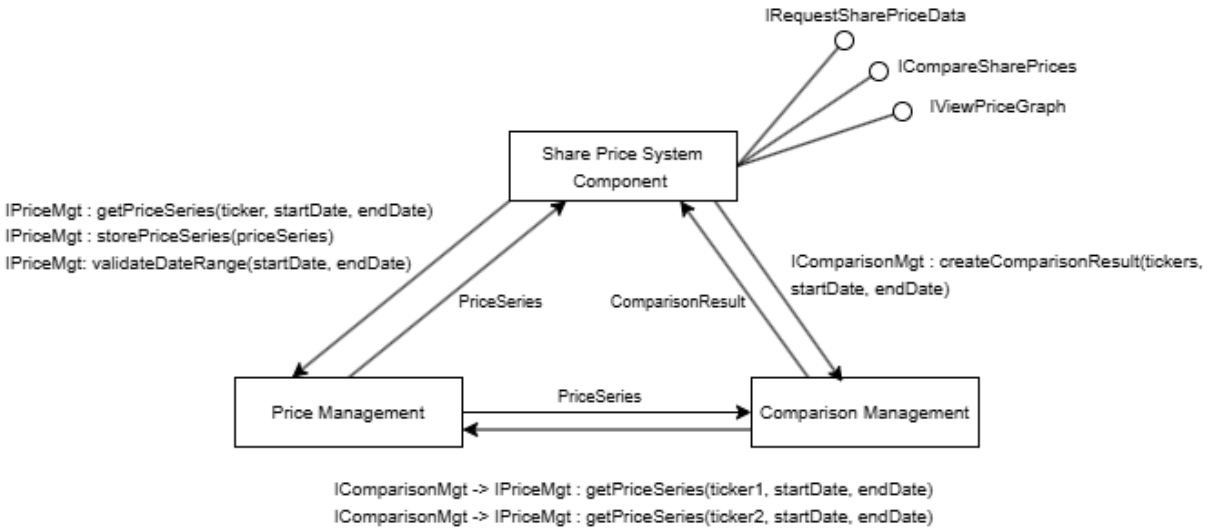
Operation	Parameters	Return Type	Purpose
getPriceSeries	ticker, startDate, endDate	PriceSeries	Obtains the price series for a given ticker over a specified date range
storePriceSeries	priceSeries	void	Stores the retrieved price series for later offline use
validateDateRange	startDate, endDate	void	As the date range is a business rule, this business interface is needed

IComparisonMgt

This interface is responsible for the comparison region centred on ComparisonResult.

Operation	Parameters	Return Type	Purpose
createComparisonResult	tickers, startDate, endDate	Comparison Result	Creates a comparison result for one or two requested tickers over the specified date range

Collaboration Diagram



Software Architecture and Design: Sprint 2

Initial System Architecture

The initial system architecture was derived from the system interfaces and business interfaces, resulting in three primary components:

- System Component
- Price Management Component
- Comparison Management Component

The System Component implements the system interfaces (IRequestSharePriceData, ICompareSharePrices, IViewPriceGraph). These are grouped together as they are user-facing operations driven by the same actor and are likely to evolve together.

The Price Management Component implements IPriceMgt and is responsible for managing price-series data, including PriceSeries, Ticker, DateRange, and PricePoint. These elements form a cohesive responsibility region centred on price data.

The Comparison Management Component implements IComparisonMgt and is responsible for producing ComparisonResult instances from one or two price series.

A dependency exists from the Comparison Management Component to the Price Management Component, as comparison operations require price-series data. The dependency is unidirectional, with price management remaining independent.

Architectural Principles

The architecture satisfies the Acyclic Dependencies Principle, as all dependencies form a directed acyclic structure: the System Component depends on both business components, and the Comparison Management Component depends on the Price Management Component, with no cycles present.

It also adheres to the Dependency Inversion Principle, as components depend on interfaces (IPriceMgt, IComparisonMgt) rather than concrete implementations. This ensures that high-level components remain independent of lower-level details.

A clear separation of concerns is maintained:

- the System Component handles orchestration,
- the Price Management Component handles data management,
- the Comparison Management Component handles comparison logic.

The grouping of price-related types reflects the Common Reuse Principle, as these elements are typically used together when processing share price data.